🧩 Chapter 8: Secure Software Development

CompTIA Security+ Study Slides

Walailak University — Assist. Prof. Dr. CJ

🎯 Learning Objectives

- Understand the **Secure Software Development Life Cycle (SDLC)**
- Explain **security principles in software design**
- Identify **secure coding practices** and **common vulnerabilities**
- Describe **testing methods** for secure applications
- Apply **DevSecOps** and **agile security integration**

🧭 What is Secure Software Development?

Definition:

Secure Software Development ensures that security is **embedded at every phase** of the SDLC — not added after deployment.

🎯 **Goal:** Deliver software that maintains **Confidentiality, Integrity, and Availability (CIA)** while minimizing vulnerabilities and risks.

🧱 The Secure Software Development Life Cycle (SDLC)

Phase	Key Security Activities
1. Planning	Identify compliance, threat landscape, and security objectives
2. Design	Apply threat modeling, architecture reviews
3. Implementation	Write secure code, use trusted libraries
4. Testing	Conduct static/dynamic analysis, penetration testing
5. Deployment	Harden environments, manage secrets
6. Maintenance	Patch vulnerabilities, monitor, audit logs

🧩 Agile and DevSecOps Integration

Agile Development

- Delivers functionality in **short iterations** (sprints)
- Security tasks embedded within each sprint
- Continuous feedback and adaptation

DevSecOps

- Integrates **Security + Development + Operations**
- "**Shift Left**" approach — apply security early in CI/CD pipeline
- Automate scanning, testing, and compliance checks

💡 *Security automation improves speed and consistency.*

🧠 Core Principles of Secure Development

- **Confidentiality:** Protect data access and transmission
- **Integrity:** Prevent unauthorized modification
- **Availability:** Ensure system reliability
- **Least Privilege:** Restrict user and process permissions
- **Defense in Depth:** Multiple layers of security
- **Never Trust User Input:** Validate and sanitize all inputs
- **Fail Securely:** Handle errors gracefully and safely

🧩 Threat Modeling

Purpose: identify potential threats, vulnerabilities, and attack paths before coding.

Steps:

1. Identify assets and entry points
2. Define trust boundaries
3. Identify threats using **STRIDE**:
    - Spoofing
    - Tampering
    - Repudiation
    - Information Disclosure
    - Denial of Service
    - Elevation of Privilege

💻 Secure Coding Practices

Practice	Description
Input Validation	Enforce data types, length, and range checks
Output Encoding	Prevent XSS by sanitizing browser output
Parameterized Queries	Avoid SQL/LDAP injection
Error Handling	Generic messages; no sensitive data leaks
Code Signing	Ensure authenticity and integrity of software
Use Trusted SDKs/Libraries	Avoid unverified third-party code
Secure Defaults	Configure secure-by-default installation options

🔒 Authentication and Access Control

- Enforce **Multi-Factor Authentication (MFA)**
- Implement **Role-Based Access Control (RBAC)**
- Apply **Session Management**:
  - Use unique session IDs
  - Set idle timeouts
  - Secure cookies (`HttpOnly`, `Secure`, `SameSite`)
- Avoid storing passwords in plaintext; hash with **bcrypt** or **Argon2**

⚙️ Defense in Depth

Layering security controls improves resilience.

Examples:

- Firewall + WAF + IDS/IPS
- Code signing + integrity checks
- Encryption + key management
- Segmentation + IAM policies

💡 One layer failing should not compromise the entire system.

📄 Input Validation and Output Sanitization

- Validate *everything coming in*
- Sanitize *everything going out*
- Reject invalid data, not sanitize after acceptance
- Whitelist preferred over blacklist

Example (Python):

```
if not re.match("^[a-zA-Z0-9_]{3,15}$", username):
    raise ValueError("Invalid username")
```

🧰 Common Software Vulnerabilities

Category	Example	Mitigation
Injection	SQL, LDAP, XML injection	Parameterized queries
Buffer Overflow	Writing beyond memory limit	Bounds checking
Cross-Site Scripting (XSS)	Script injection	Output encoding
Cross-Site Request Forgery (CSRF)	Forged authenticated request	Anti-CSRF tokens
Race Condition	Exploiting timing flaws	Synchronization locks
Privilege Escalation	Gaining higher rights	Enforce least

🧪 Testing and Verification

Static Analysis (SAST)

- Analyze source code without execution
- Detects coding flaws early (SonarQube, Bandit)

Dynamic Analysis (DAST)

- Tests running applications (OWASP ZAP, Burp Suite)

Fuzz Testing

- Injects random data to detect errors and crashes

Penetration Testing

- Simulates real-world attack scenarios

🧩 White-box, Black-box, and Gray-box Testing

Type	Tester Knowledge	Purpose
White-box	Full access to source and architecture	Code quality and logic verification
Black-box	No internal knowledge	Realistic external attack simulation
Gray-box	Partial knowledge	Combines both perspectives

💡 Combine multiple methods for comprehensive coverage.

Error and Exception Handling

- Do **not** display detailed error messages to users
- Log errors securely (use centralized logging system)
- Fail securely and preserve system state
- Validate input even when errors occur
- Implement **graceful degradation** under failure conditions

🧬 Secure Software Testing Tools

- **Static Analysis Tools:** SonarQube, Checkmarx, Fortify
- **Dynamic Testing Tools:** OWASP ZAP, Burp Suite
- **Fuzzers:** Peach, AFL, Boofuzz
- **Dependency Scanners:** Snyk, Dependency-Check
- **Continuous Integration Security:** GitHub Advanced Security, GitLab SAST/DAST

🔑 Secure Deployment and Maintenance

- Deploy only on **hardened servers**
- Use **sandboxing** and **container isolation**
- Apply **version control and code integrity checks**
- Rotate keys, credentials, and tokens regularly
- Conduct regular **vulnerability scanning** and **patching**

🧩 Software Vulnerability Management

1. **Identify** – find vulnerabilities (scanners, bug reports)
2. **Assess** – rate severity using **CVSS**
3. **Remediate** – patch or fix code
4. **Validate** – retest and verify resolution
5. **Monitor** – track new vulnerabilities continuously

🧱 Secure SDLC Automation (CI/CD)

- Integrate **security scanning tools** into CI/CD pipelines
- Automate:
  - Dependency scanning
  - Code analysis
  - Policy enforcement
- Enable automated rollback if deployment fails security checks

💡 *Automated security improves consistency and developer velocity.*

📜 OWASP Secure Coding Practices

- Validate input & encode output
- Enforce authentication and access control
- Store sensitive data securely
- Implement secure error handling
- Log all security events
- Keep components up to date

Reference: OWASP Secure Coding Practices Guide

🧠 Key Takeaways

- Secure SDLC integrates security across all development phases
- Apply **threat modeling**, **secure coding**, and **continuous testing**
- Automate security in DevSecOps pipelines
- Regularly train developers on **OWASP** and secure frameworks
- Continuous monitoring ensures long-term software resilience

🏁 End of Chapter 8

Next: Chapter 9 — Network Design & Security Architecture